LabLink

Database Design and Implementation

CS450 Software Engineering - Week 11 Team Project Deliverable
Professor Sean Perryman - Campbellsville University

Prepared by Weston T. Mattingly for the LabLink team - November 3, 2026

Repository: https://github.com/westonm05/Reservation-Project

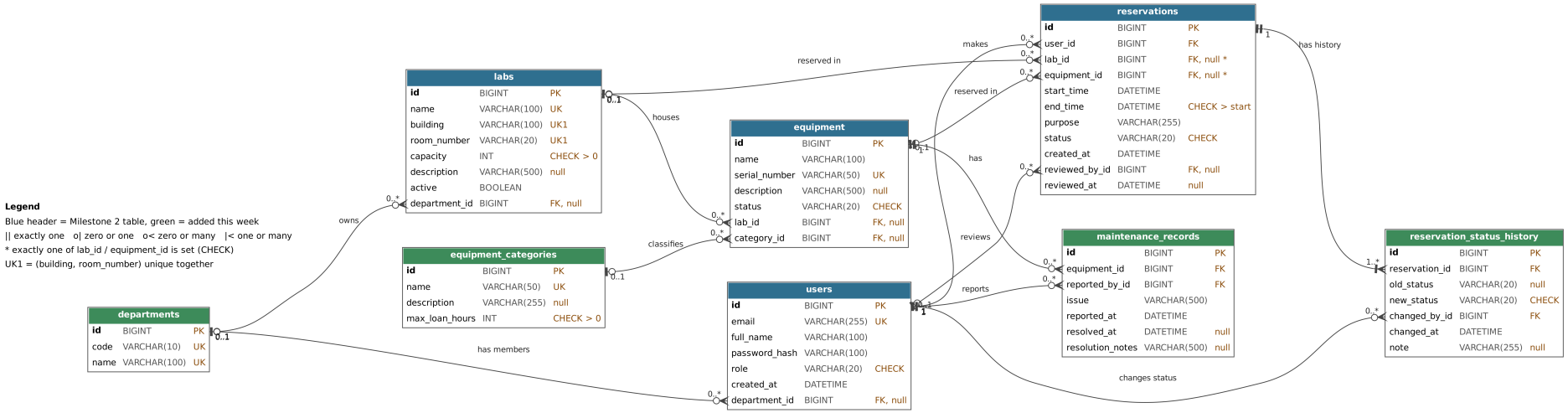
This week's work (branch): github.com/westonm05/Reservation-Project/tree/feature/week11-database

Deliverables

Requirement	Section	Location in the repository
1. ER diagram, 6+ entities, cardinality	1	docs/diagrams/er-diagram.svg (8 entities, 12 relationships)
2. Normalized to 3NF, denormalization justified	2	docs/database-design.md
3. SQL DDL: tables, constraints, FKs, indexes	3	src/main/resources/db/migration/V1__ ... V4__
4. ORM model classes	4	src/main/java/.../lablink/model/ and .../repository/ (JPA / Hibernate)
5. Migration scripts in Git	5	Flyway: src/main/resources/db/migration/
6. Seed data, 20+ rows per major table	6	src/main/resources/db/seed/R__seed_test_data.sql

Stack: Spring Boot 3.5, Spring Data JPA (Hibernate ORM), Flyway, MySQL 8 in production, H2 (MySQL mode) for development and automated tests. All 94 automated tests pass in GitHub Actions on this branch, including a new integration test that checks the migrations, the seed data and the ORM mappings.

1. Entity-Relationship Diagram



Crow's-foot notation, with the cardinality also written at each end of every line. Blue tables existed in Milestone 2; the four green tables were added this week. Each box lists column, type and key/constraint.

Relationships and cardinality

#	Relationship	Cardinality	Implemented by
1	Department **has members** User	0..1 : 0..*	users.department_id (nullable - self-registered students may not choose one)
2	Department **owns** Lab	0..1 : 0..*	labs.department_id
3	Lab **houses** Equipment	0..1 : 0..*	equipment.lab_id (null = portable)
4	EquipmentCategory **classifies** Equipment	0..1 : 0..*	equipment.category_id
5	User **makes** Reservation	1 : 0..*	reservations.user_id (NOT NULL)
6	User **reviews** Reservation	0..1 : 0..*	reservations.reviewed_by_id
7	Lab **reserved in** Reservation	0..1 : 0..*	reservations.lab_id
8	Equipment **reserved in** Reservation	0..1 : 0..*	reservations.equipment_id; CHECK: exactly one of 7 or 8
9	Reservation **has history** ReservationStatusChange	1 : 1..*	reservation_status_history.reservation_id; "at least one" is guaranteed by ReservationService, which writes a row when the reservation is created
10	User **changes status** ReservationStatusChange	1 : 0..*	reservation_status_history.changed_by_id
11	Equipment **has** MaintenanceRecord	1 : 0..*	maintenance_records.equipment_id
12	User **reports** MaintenanceRecord	1 : 0..*	maintenance_records.reported_by_id

Entities

Entity / table	What one row is
User / users	A person who logs in: student, faculty member or lab manager
Department / departments	An academic department (Computer Science, Biology, ...)
Lab / labs	A reservable room
EquipmentCategory / equipment_categories	A kind of equipment and its maximum loan length
Equipment / equipment	One physical, reservable item, identified by serial number
Reservation / reservations	A request to use one lab or one item for a time window
ReservationStatusChange / reservation_status_history	One status change of a reservation: who, when, from, to
MaintenanceRecord / maintenance_records	One problem reported on an item and how it was fixed

2. Normalization Analysis

Relations

Primary keys are underlined; *italic* columns are foreign keys.

- **departments**(id, code, name)
- **users**(id, email, full_name, password_hash, role, created_at, *department_id*)
- **labs**(id, name, building, room_number, capacity, description, active, *department_id*)
- **equipment_categories**(id, name, description, max_loan_hours)
- **equipment**(id, name, serial_number, description, status, *lab_id*, *category_id*)
- **reservations**(id, *user_id*, *lab_id*, *equipment_id*, start_time, end_time, purpose, status, created_at, *reviewed_by_id*, reviewed_at)
- **reservation_status_history**(id, *reservation_id*, old_status, new_status, *changed_by_id*, changed_at, note)
- **maintenance_records**(id, *equipment_id*, *reported_by_id*, issue, reported_at, resolved_at, resolution_notes)

Functional dependencies

Relation	Dependencies (determinant -> dependents)	Candidate keys
departments	id -> code, name; code -> id; name -> id	id, code, name
users	id -> all other columns; email -> id	id, email
labs	id -> all other columns; name -> id; (building, room_number) -> id	id, name, (building, room_number)
equipment_categories	id -> name, description, max_loan_hours; name -> id	id, name
equipment	id -> all other columns; serial_number -> id	id, serial_number
reservations	id -> all other columns	id
reservation_status_history	id -> all other columns	id
maintenance_records	id -> all other columns	id

Normal form check

First Normal Form. Every column holds one atomic value and every table has a primary key. Nothing is stored as a list: a lab's equipment, a reservation's history and an item's repair log are separate tables linked by foreign keys, not comma-separated text.

Second Normal Form. Every primary key is a single surrogate column (id), so no column can depend on only part of a key.

Third Normal Form. No non-key column depends on another non-key column. Every determinant in the table above is a candidate key, so the schema also satisfies **Boyce-Codd Normal Form**.

Normalization decisions made this week

The Milestone 2 schema was already in 3NF. Each new requirement could have been added as extra columns on existing tables; every one of those would have broken 3NF, so each became its own table:

If we had stored...	Problem it creates	What we did instead
users.department_name and labs.department_name	department_code -> department_name is a non-key dependency, repeated in every user and lab row	departments table; users and labs keep only department_id
equipment.category_name and equipment.max_loan_hours	category_name -> max_loan_hours: changing the laptop limit means updating every laptop row	equipment_categories; the limit is stored once per category
equipment.last_issue, equipment.last_repaired_at	A repeating group (an item has many repairs); only the latest could be kept	maintenance_records, one row per issue
reservations.approved_by_name	reviewed_by_id -> reviewer name (transitive)	store reviewed_by_id only; the name comes from users

Anomalies avoided. In an unnormalized equipment table with columns (serial, item, category, max_loan_hours, lab, building), max_loan_hours depends on category and building depends on lab. Changing the laptop limit or moving a lab means editing many rows (update anomaly); a new category cannot be recorded until an item of that kind exists (insertion anomaly); and deleting the last laptop also deletes the laptop loan limit (deletion anomaly).

Intentional denormalization

Two places deliberately store values that could be derived from reservation_status_history:

1. reservations.status. The current status always equals the new_status of the reservation's latest history row. *Justification:* the double-booking check runs on every new or rescheduled reservation and filters on status IN ('PENDING', 'APPROVED'), and the approval queue filters on status = 'PENDING'. Deriving the status would turn both into a "latest row per reservation" subquery over the history table; keeping the column keeps them simple and indexable (idx_reservations_status_start). *Consistency:* only ReservationService changes a status, and it updates the column and inserts the history row in the same database transaction.

2. reservations.reviewed_by_id and reviewed_at. These repeat changed_by_id and changed_at of the APPROVED or REJECTED history row. *Justification:* the Manage Reservations page shows the reviewer for every row; two columns avoid a join and filter on history per row, and existing pages and tests already use them. *Consistency:* written in the same review() call that writes the history row.

Not denormalization: role and status values are VARCHAR columns with CHECK constraints rather than lookup tables because the value sets are fixed in code; and one reservations table with two nullable foreign keys (plus a CHECK) replaces two separate tables so "my reservations" and the approval queue remain single-table queries.

3. SQL DDL

The DDL is the four Flyway migration scripts below, run in order. They use only SQL that both MySQL 8 and H2 (MySQL mode) accept, so tests run against exactly the production schema.

Constraint type	Where it is used
PRIMARY KEY	Every table: BIGINT AUTO_INCREMENT surrogate key
FOREIGN KEY	All 12 relationships in the ER diagram (one FK column each)
UNIQUE	users.email; labs.name; labs(building, room_number); equipment.serial_number; departments.code and name; equipment_categories.name
CHECK	role and status value lists; capacity > 0; max_loan_hours > 0; end_time > start_time; exactly one reservation target; resolved_at >= reported_at; old_status <> new_status
NOT NULL	Every required column
INDEX	Seven composite/secondary indexes in V4, each matched to a query the app runs (see V4 comments)

V1__create_core_tables.sql

```
-- V1: the four core tables from Milestone 2 (users, labs, equipment, reservations).
--
-- Flyway runs every file in db/migration in version order (V1, V2, ...) exactly once and
-- records it in the flyway_schema_history table. Never edit a migration after it has been
-- run on a shared database: add a new V<n> file instead.
--
-- Written in SQL that both MySQL 8 (production) and H2 in MySQL mode (development/tests) accept.

CREATE TABLE users (
    id            BIGINT       NOT NULL AUTO_INCREMENT,
    email         VARCHAR(255) NOT NULL,
    full_name     VARCHAR(100) NOT NULL,
    password_hash VARCHAR(100) NOT NULL,
    role          VARCHAR(20)  NOT NULL,
    created_at    DATETIME     NOT NULL,
    CONSTRAINT pk_users PRIMARY KEY (id),
    CONSTRAINT uq_users_email UNIQUE (email),
    CONSTRAINT chk_users_role CHECK (role IN ('STUDENT', 'FACULTY', 'LAB_MANAGER'))
);

CREATE TABLE labs (
    id          BIGINT       NOT NULL AUTO_INCREMENT,
    name        VARCHAR(100) NOT NULL,
    building    VARCHAR(100) NOT NULL,
    room_number VARCHAR(20)  NOT NULL,
    capacity    INT          NOT NULL,
    description VARCHAR(500),
    active      BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_labs PRIMARY KEY (id),
    CONSTRAINT uq_labs_name UNIQUE (name),
    CONSTRAINT uq_labs_location UNIQUE (building, room_number),
    CONSTRAINT chk_labs_capacity CHECK (capacity > 0)
);

CREATE TABLE equipment (
    id            BIGINT       NOT NULL AUTO_INCREMENT,
    name          VARCHAR(100) NOT NULL,
    serial_number VARCHAR(50)  NOT NULL,
    description   VARCHAR(500),
    status        VARCHAR(20)  NOT NULL,
    lab_id        BIGINT,
    CONSTRAINT pk_equipment PRIMARY KEY (id),
    CONSTRAINT uq_equipment_serial UNIQUE (serial_number),
    CONSTRAINT fk_equipment_lab FOREIGN KEY (lab_id) REFERENCES labs (id),
    CONSTRAINT chk_equipment_status CHECK (status IN ('AVAILABLE', 'MAINTENANCE', 'RETIRED'))
);

CREATE TABLE reservations (
    id             BIGINT       NOT NULL AUTO_INCREMENT,
    user_id        BIGINT       NOT NULL,
    lab_id         BIGINT,
```

```
    equipment_id   BIGINT,
    start_time     DATETIME     NOT NULL,
    end_time       DATETIME     NOT NULL,
    purpose        VARCHAR(255) NOT NULL,
    status         VARCHAR(20)  NOT NULL,
    created_at     DATETIME     NOT NULL,
    reviewed_by_id BIGINT,
    reviewed_at    DATETIME,
    CONSTRAINT pk_reservations PRIMARY KEY (id),
    CONSTRAINT fk_reservations_user FOREIGN KEY (user_id) REFERENCES users (id),
    CONSTRAINT fk_reservations_lab FOREIGN KEY (lab_id) REFERENCES labs (id),
    CONSTRAINT fk_reservations_equipment FOREIGN KEY (equipment_id) REFERENCES equipment (id),
    CONSTRAINT fk_reservations_reviewer FOREIGN KEY (reviewed_by_id) REFERENCES users (id),
    CONSTRAINT chk_reservations_times CHECK (end_time > start_time),
    CONSTRAINT chk_reservations_status CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED')),
    -- a reservation is for exactly one lab OR one piece of equipment
    CONSTRAINT chk_reservations_target CHECK (
        (lab_id IS NOT NULL AND equipment_id IS NULL) OR
        (lab_id IS NULL AND equipment_id IS NOT NULL))
);
```

V2__add_departments_and_equipment_categories.sql

```
-- V2: two lookup tables, linked to existing tables with new foreign-key columns.
--
-- departments          - academic departments; a user belongs to one, a lab is owned by one
-- equipment_categories - kinds of equipment (laptop, 3D printer...) and their loan limit
--
-- The new foreign-key columns are nullable so rows created before this migration stay valid
-- and so self-registered students don't have to pick a department.

CREATE TABLE departments (
    id   BIGINT       NOT NULL AUTO_INCREMENT,
    code VARCHAR(10)  NOT NULL,
    name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_departments PRIMARY KEY (id),
    CONSTRAINT uq_departments_code UNIQUE (code),
    CONSTRAINT uq_departments_name UNIQUE (name)
);

CREATE TABLE equipment_categories (
    id             BIGINT       NOT NULL AUTO_INCREMENT,
    name           VARCHAR(50)  NOT NULL,
    description    VARCHAR(255),
    max_loan_hours INT          NOT NULL,
    CONSTRAINT pk_equipment_categories PRIMARY KEY (id),
    CONSTRAINT uq_equipment_categories_name UNIQUE (name),
    CONSTRAINT chk_equipment_categories_hours CHECK (max_loan_hours > 0)
);

ALTER TABLE users ADD COLUMN department_id BIGINT;
ALTER TABLE users ADD CONSTRAINT fk_users_department
    FOREIGN KEY (department_id) REFERENCES departments (id);

ALTER TABLE labs ADD COLUMN department_id BIGINT;
ALTER TABLE labs ADD CONSTRAINT fk_labs_department
    FOREIGN KEY (department_id) REFERENCES departments (id);

ALTER TABLE equipment ADD COLUMN category_id BIGINT;
ALTER TABLE equipment ADD CONSTRAINT fk_equipment_category
    FOREIGN KEY (category_id) REFERENCES equipment_categories (id);
```

V3__add_maintenance_records_and_status_history.sql

```
-- V3: two history tables.
--
-- maintenance_records         - problems reported on a piece of equipment and how they were fixed
-- reservation_status_history  - one row every time a reservation changes status (an audit trail:
--                               who submitted, approved, rejected or cancelled it, and when)

CREATE TABLE maintenance_records (
    id              BIGINT       NOT NULL AUTO_INCREMENT,
    equipment_id    BIGINT       NOT NULL,
    reported_by_id  BIGINT       NOT NULL,
    issue           VARCHAR(500) NOT NULL,
    reported_at     DATETIME     NOT NULL,
    resolved_at     DATETIME,
```

```
    resolution_notes VARCHAR(500),
    CONSTRAINT pk_maintenance_records PRIMARY KEY (id),
    CONSTRAINT fk_maintenance_equipment FOREIGN KEY (equipment_id) REFERENCES equipment (id),
    CONSTRAINT fk_maintenance_reporter FOREIGN KEY (reported_by_id) REFERENCES users (id),
    -- an open record has no resolved_at; a closed one can't be resolved before it was reported
    CONSTRAINT chk_maintenance_dates CHECK (resolved_at IS NULL OR resolved_at >= reported_at)
);

CREATE TABLE reservation_status_history (
    id             BIGINT       NOT NULL AUTO_INCREMENT,
    reservation_id BIGINT       NOT NULL,
    old_status     VARCHAR(20),          -- NULL on the row written when the reservation is created
    new_status     VARCHAR(20)  NOT NULL,
    changed_by_id  BIGINT       NOT NULL,
    changed_at     DATETIME     NOT NULL,
    note           VARCHAR(255),
    CONSTRAINT pk_reservation_status_history PRIMARY KEY (id),
    CONSTRAINT fk_history_reservation FOREIGN KEY (reservation_id) REFERENCES reservations (id),
    CONSTRAINT fk_history_changed_by FOREIGN KEY (changed_by_id) REFERENCES users (id),
    CONSTRAINT chk_history_old_status CHECK (
        old_status IS NULL OR old_status IN ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED')),
    CONSTRAINT chk_history_new_status CHECK (new_status IN ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED')),
    CONSTRAINT chk_history_changes CHECK (old_status IS NULL OR old_status <> new_status)
);
```

V4__add_query_indexes.sql

```
-- V4: indexes for the queries the application runs most often.
--
-- Primary keys and UNIQUE constraints already have indexes. MySQL also creates an index for a
-- foreign key automatically, but only on that one column; the composite indexes below cover the
-- whole WHERE / ORDER BY of each query, and MySQL reuses them for the foreign key as well
-- (the foreign-key column is the first column of each index).

-- Double-booking check (ADR-0004), run on every new or rescheduled reservation:
--   WHERE lab_id = ? AND status IN ('PENDING','APPROVED') AND start_time < ? AND end_time > ?
CREATE INDEX idx_reservations_lab_time       ON reservations (lab_id, start_time, end_time);
CREATE INDEX idx_reservations_equipment_time ON reservations (equipment_id, start_time, end_time);

-- Lab manager's approval queue:  WHERE status = 'PENDING' ORDER BY start_time
CREATE INDEX idx_reservations_status_start   ON reservations (status, start_time);

-- "My reservations" page:  WHERE user_id = ? ORDER BY start_time DESC
CREATE INDEX idx_reservations_user_start     ON reservations (user_id, start_time);

-- Equipment list page:  ORDER BY name
CREATE INDEX idx_equipment_name              ON equipment (name);

-- A reservation's audit trail, oldest first:  WHERE reservation_id = ? ORDER BY changed_at
CREATE INDEX idx_history_reservation_time    ON reservation_status_history (reservation_id, changed_at);

-- Open maintenance tickets for one item:  WHERE equipment_id = ? AND resolved_at IS NULL
CREATE INDEX idx_maintenance_equipment_open  ON maintenance_records (equipment_id, resolved_at);
```

4. ORM: Spring Data JPA / Hibernate

Each table maps to one `@Entity` class in `edu.campbellsville.lablink.model` and one Spring Data repository interface in `...lablink.repository`. Hibernate runs with `spring.jpa.hibernate.ddl-auto=validate`: it never creates tables, but the application refuses to start if an entity does not match the tables Flyway built, so the model classes and the DDL cannot drift apart.

Table	Entity class	Relationships mapped	Repository
departments	Department	-	DepartmentRepository
users	User	@ManyToOne department	UserRepository
labs	Lab	@ManyToOne department	LabRepository
equipment_categories	EquipmentCategory	-	EquipmentCategoryRepository
equipment	Equipment	@ManyToOne lab, category	EquipmentRepository
reservations	Reservation	@ManyToOne user, lab, equipment, reviewedBy; @OneToMany(mappedBy) history	ReservationRepository
reservation_status_history	ReservationStatusChange	@ManyToOne(LAZY) reservation, changedBy	ReservationStatusChangeRepository
maintenance_records	MaintenanceRecord	@ManyToOne(LAZY) equipment, reportedBy	MaintenanceRecordRepository

The ORM is used by the application, not only declared: ReservationService writes a status-history row on every submit, approve, reject and cancel, and rejects equipment reservations longer than the item's category max_loan_hours; the lab form now has a department dropdown and the equipment form a category dropdown.

Example entity: ReservationStatusChange

```
/**
 * One row of a reservation's audit trail: "on [changedAt], [changedBy] moved it from
 * [oldStatus] to [newStatus]". Written by ReservationService and never changed afterwards.
 * oldStatus is null for the row written when the reservation is first created.
 */
@Entity
@Table(name = "reservation_status_history")
public class ReservationStatusChange {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(name = "reservation_id", nullable = false)
    private Reservation reservation;

    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.VARCHAR)
    @Column(name = "old_status", length = 20)
    private ReservationStatus oldStatus;

    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.VARCHAR)
    @Column(name = "new_status", nullable = false, length = 20)
    private ReservationStatus newStatus;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(name = "changed_by_id", nullable = false)
    private User changedBy;

    @Column(name = "changed_at", nullable = false)
    private LocalDateTime changedAt = LocalDateTime.now();

    private String note;
    // constructors and getters omitted - see the repository
}
```

Example repository: ReservationStatusChangeRepository

```
public interface ReservationStatusChangeRepository extends JpaRepository<ReservationStatusChange, Long> {

    /** A reservation's audit trail, oldest first (id breaks ties within the same second). */
    List<ReservationStatusChange> findByReservationOrderByChangedAtAscIdAsc(Reservation reservation);
}
```

Spring Data generates the SQL for this method from its name: SELECT ... WHERE reservation_id = ? ORDER BY changed_at, id - served by idx_history_reservation_time.

5. Database Migrations (Flyway)

Flyway runs automatically when the application starts. It applies each versioned script exactly once, in version order, and records it in the `flyway_schema_history` table, so an empty database is brought to the current schema with one command and an up-to-date one is left alone.

Script	Purpose
V1__create_core_tables.sql	users, labs, equipment, reservations (the Milestone 2 schema)
V2__add_departments_and_equipment_categories.sql	Two lookup tables; ALTER TABLE adds department_id and category_id foreign keys to users, labs and equipment
V3__add_maintenance_records_and_status_history.sql	The two history tables
V4__add_query_indexes.sql	Composite indexes for the main queries
db/seed/R__seed_test_data.sql	Repeatable seed script (section 6); runs after the versioned scripts

Running the migrations from scratch

```
# Development: H2 in memory - schema + seed data are rebuilt on every start
mvn spring-boot:run

# Production: an empty MySQL database, schema only
DB_URL=jdbc:mysql://localhost:3306/lablink DB_USERNAME=lablink DB_PASSWORD=... \
  mvn spring-boot:run -Dspring-boot.run.profiles=mysql

# Production schema plus the demo data
mvn spring-boot:run -Dspring-boot.run.profiles=mysql,seed
```

Configuration

```
spring.application.name=lablink

# ---- Development database: H2, in memory, recreated every time the app starts ----
spring.datasource.url=jdbc:h2:mem:lablink;MODE=MySQL;DATABASE_TO_LOWER=TRUE;DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.datasource.password=

# ---- Schema: Flyway migrations (see docs/database-design.md and ADR-0005) ----
# db/migration holds the schema (V1__, V2__ ... run once each, in order).
# db/seed holds R__seed_test_data.sql, the demo/test data. It is included here for development
# and tests only; the mysql profile leaves it out unless the "seed" profile is also active.
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration,classpath:db/seed

# Hibernate never creates or changes tables. "validate" makes startup fail if an entity class
# doesn't match the tables Flyway built, so the ORM model and the DDL can't drift apart.
spring.jpa.hibernate.ddl-auto=validate

spring.jpa.open-in-view=true
spring.thymeleaf.cache=false
```

Team rules

- Never edit a migration after it has run on a shared database; add V5__... instead.
- A MySQL database created from the old Milestone 2 schema.sql is handled with baseline-on-migrate at version 1: Flyway treats V1 as already applied and continues from V2.
- The decision is recorded in docs/adr/0005-flyway-migrations-and-sql-seed-data.md.

6. Seed Data

`R__seed_test_data.sql` is a Flyway *repeatable* migration: it runs after the versioned scripts and again whenever the file changes. It deletes all rows first (children before parents), so running it again always produces the same data, and it looks up foreign keys by natural key (email, lab name, serial number) rather than hard-coded ids, so it works with any auto-increment values. It is loaded for development and tests, and in production only when the seed profile is added.

Table	Rows	Contents
users	25	3 lab managers, 6 faculty, 16 students (including the three demo logins)
labs	20	Across 6 buildings; 2 inactive
equipment	30	10 categories; 3 in MAINTENANCE, 2 RETIRED
reservations	40	10 pending, 22 approved, 3 rejected, 5 cancelled; past and upcoming; no overlapping active bookings
reservation_status_history	62	Matches each reservation's status, reviewer and review time
maintenance_records	21	3 open (one per MAINTENANCE item), 18 resolved
departments (lookup)	8	One row per department
equipment_categories (lookup)	10	One row per equipment type

The six major tables hold at least 20 rows each. The two lookup tables contain one row per real department or equipment type; padding them to 20 would mean inventing departments that don't exist.

How the seed data was checked

- Loaded twice in a row into a test database to confirm it re-runs cleanly; foreign-key, CHECK and UNIQUE constraints all pass.
- Consistency queries: no two pending/approved bookings of one lab or item overlap; no future booking of an inactive lab or unavailable item; equipment bookings respect the category loan limit; every reservation's latest history row matches its status; every MAINTENANCE item has an open ticket.
- DatabaseMigrationIntegrationTest (runs in CI) asserts all four migrations applied, at least 20 rows in each major table, seeded foreign keys resolve through the ORM, and ReservationService writes history rows. CI result on this branch: 94 tests, 0 failures.

Demo logins: manager@lablink.edu / manager123, faculty@lablink.edu / faculty123, student@lablink.edu / student123; every other seeded account uses password123.

Excerpt of the seed script

```
-- Children first, parents last, so no foreign key is ever left dangling.
DELETE FROM reservation_status_history;
DELETE FROM maintenance_records;
DELETE FROM reservations;
DELETE FROM equipment;
DELETE FROM labs;
DELETE FROM users;
DELETE FROM equipment_categories;
DELETE FROM departments;

-- ---------------------------------------------------------------- departments (8)
INSERT INTO departments (code, name) VALUES
    ('CS', 'Computer Science'),
    ('IT', 'Information Technology'),
    ('MATH', 'Mathematics'),
    ('BIO', 'Biology'),
    ('CHEM', 'Chemistry'),
    ('PHYS', 'Physics'),
    ('ART', 'Art and Design'),
```

```
    ('COMM', 'Mass Communication');

-- ---------------------------------------------------------------- equipment_categories (10)
INSERT INTO equipment_categories (name, description, max_loan_hours) VALUES
    ('Laptop', 'Windows and macOS loaner laptops', 72),
    ('Tablet', 'iPads and Android tablets', 72),
    ('3D Printer', 'FDM and resin printers', 8),
    ('Raspberry Pi Kit', 'Single-board computer kits with sensors', 168),
    ('Arduino Kit', 'Microcontroller starter kits', 168),
    ('Oscilloscope', 'Digital storage oscilloscopes', 24),
    ('Microscope', 'Compound and digital microscopes', 24),

-- ... users, labs, equipment ...

-- ---------------------------------------------------------------- reservations (40)
-- Past ones are finished (approved, rejected or cancelled); future ones include pending requests
-- waiting in the lab manager's queue. No two pending/approved bookings of the same lab or item overlap.
INSERT INTO reservations (user_id, lab_id, equipment_id, start_time, end_time, purpose, status, created_at, reviewed_by_id...
    ((SELECT id FROM users WHERE email = 'faculty@lablink.edu'), NULL, (SELECT id FROM equipment WHERE serial_number = 'MI...
    ((SELECT id FROM users WHERE email = 'faculty@lablink.edu'), (SELECT id FROM labs WHERE name = 'Media Lab'), NULL, '20...
    ((SELECT id FROM users WHERE email = 'grace.okafor@lablink.edu'), (SELECT id FROM labs WHERE name = 'Computer Science ...
    -- ... 36 more rows; full file in the repository
```

Full scripts and model code: https://github.com/westonm05/Reservation-Project/tree/feature/week11-database